

THE AMDAHL REALITY GAP

Student ID: 230103170

Hardware

CPU	Apple M4
Architecture	arm64
Physical / logical CPU cores	10 / 10
Performance / efficiency cores	4 / 6

Workload and Sequential Baseline

The workload size uses the last four digits of the student ID: $N = 10,000,000 + (3170 \times 1,000) = 13,170,000$. The program computes Collatz stopping steps for all values from 1 to N. The final maximum stopping-step count was **688**, and the checksum modulo 1,000,000,007 was **82,237,874**.

Run	Time (s)	Use
Run 1	1.479557	Warm-up - discarded
Run 2	1.478827	Baseline
Run 3	1.478508	Baseline

$$T_{\text{seq}} = (1.478827 + 1.478508) / 2 = \mathbf{1.4786675 \text{ s}}$$

Q1. What parallel fraction does Amdahl's Law predict?

Using the two-thread measurement $T_2 = 0.794056 \text{ s}$, $S_2 = T_{\text{seq}} / T_2 = 1.8622$. Amdahl's Law is:

$$S(k) = 1 / ((1 - p) + p/k)$$

Solving with $k = 2$ gives $p = \mathbf{0.9260}$. Therefore, approximately **92.60%** of the workload is parallelizable and approximately **7.40%** behaves as the serial fraction according to this fit.

Q2. How does measured scaling compare with Amdahl's prediction?

Threads	Time (s)	Measured speedup
1	1.483203	0.997x
2	0.794056	1.862x
4	0.450845	3.280x
8	0.274151	5.394x
10	0.239414	6.176x
16	0.238670	6.196x

Measured speedup rises strongly up to the physical core count but does not continue to scale linearly. The 16-thread result is almost identical to the 10-thread result because the machine has 10 physical and 10 logical CPU cores. Real execution also includes OpenMP overhead, thread management, scheduling, load imbalance, cache effects, and the heterogeneous 4-performance-core plus 6-efficiency-core architecture. These effects create the reality gap between the idealized Amdahl model and measured performance.

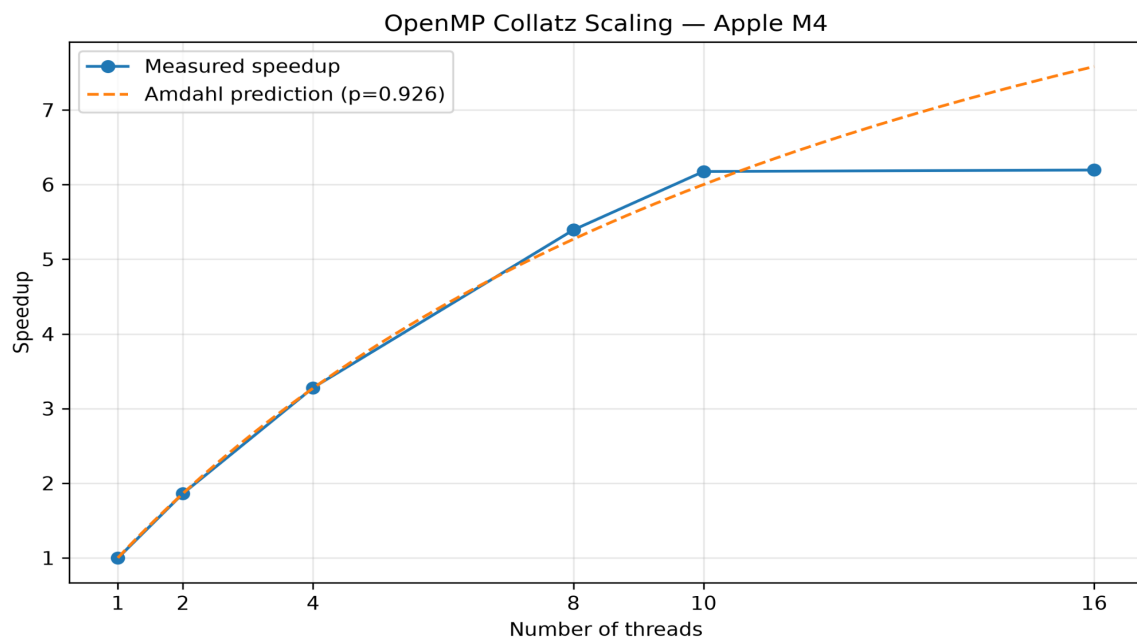


Figure 1. Measured OpenMP speedup compared with the Amdahl prediction fitted from the two-thread result.

Q3. What was the effect of false sharing?

Variant	Time (s)	Result
Adjacent counters	0.297862	2.172x vs padded
Padded counters	0.137113	Reference

Both versions produced the same checksum of 1,317,000,000. The adjacent-counter version was approximately **2.17 times slower** than the padded version. Padding separates per-thread counters in memory and reduces cache-line contention, demonstrating a significant false-sharing penalty in this benchmark.

Q4. How did OpenMP scheduling affect performance?

Schedule	Time (s)
static	0.234338
static,1000	0.218408
dynamic,100	0.219916
dynamic,10000	0.227578
guided	0.214514

All variants produced max_steps = 688 and checksum = 82,237,874. **Guided** scheduling was fastest at **0.214514 s**, approximately **8.5% faster** than the default static result in this run. Collatz iterations have variable execution costs, so an adaptive scheduling strategy can improve load balance. In this measurement, guided scheduling provided the best balance between scheduling overhead and workload distribution.

Conclusion

Parallel execution substantially reduced the Collatz runtime on the Apple M4. The fitted parallel fraction was approximately **92.60%**. The best measured scaling result was approximately **6.20x** at 16 OpenMP threads, although performance had already nearly plateaued at the machine's 10-core hardware limit. The experiments also show that practical multicore performance depends on more than the theoretical parallel fraction: cache-line contention, scheduling policy, OpenMP overhead, load imbalance, and hardware core characteristics all contribute to the Amdahl reality gap.